

UNIVERSIDADE DE BRASÍLIA
INSTITUTO DE CIÊNCIAS EXATAS
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

116394 – ORGANIZAÇÃO E ARQUITETURA DE COMPUTADORES

Relatório de Implementação
Jogo da Vida em Assembly RISC-V

Aluno: Victor Enéias

Matrícula: 221038364

Turma: Turma 2

Professor: RICARDO JACOBI

1. Configurações adotadas no RARS

O programa foi desenvolvido em linguagem Assembly RISC-V utilizando o simulador RARS. Para a exibição gráfica da simulação, foi utilizada a ferramenta *Bitmap Display*, disponível no menu *Tools* do RARS.

A matriz utilizada no programa possui dimensão 16×16 , totalizando 256 células. Cada célula foi armazenada como um byte na memória, usando os seguintes valores:

- 0: ausência de indivíduo, ou célula morta;
- 1: presença de indivíduo, ou célula viva.

Foram utilizadas duas matrizes:

- `mat1`: matriz inicial, definida diretamente na área `.data`;
- `mat2`: matriz auxiliar, usada para armazenar a próxima geração.

A configuração adotada no *Bitmap Display* foi:

- **Unit Width in Pixels:** 8

- **Unit Height in Pixels:** 8
- **Display Width in Pixels:** 128
- **Display Height in Pixels:** 128
- **Base address for display:** 0x10040000 (heap)

Com essa configuração, cada célula da matriz 16×16 é representada graficamente por um bloco de 8×8 pixels. Assim, a matriz completa ocupa uma janela gráfica de 128×128 pixels.

No código, o endereço inicial do display foi definido por meio da constante:

```
.eqv DISPLAY_BASE 0x10040000
```

Também foram definidas duas constantes para representar as cores utilizadas na tela:

```
.eqv COR_MORTA      0x00000000
.eqv COR_VIVA       0x00FFFF00
```

A cor `COR_MORTA` representa células mortas, exibidas em preto. A cor `COR_VIVA` representa células vivas, exibidas em amarelo.

2. Descrição geral do programa

O programa implementa o Jogo da Vida, um autômato celular bidimensional proposto por John Conway. Cada posição da matriz representa uma célula, que pode estar viva ou morta. A cada geração, o estado da matriz é atualizado de acordo com a quantidade de vizinhos vivos de cada célula.

As regras implementadas foram:

1. Uma célula morta nasce se possuir exatamente 3 vizinhos vivos.
2. Uma célula viva sobrevive se possuir 2 ou 3 vizinhos vivos.
3. Uma célula viva morre se possuir menos de 2 vizinhos vivos, por solidão.
4. Uma célula viva morre se possuir mais de 3 vizinhos vivos, por excesso de vizinhos.
5. Células fora dos limites da matriz são consideradas mortas.

A matriz inicial é armazenada em `mat1`. A matriz `mat2` é usada como matriz auxiliar para armazenar a próxima geração. O programa alterna o uso das matrizes, calculando ora `mat2` a partir de `mat1`, ora `mat1` a partir de `mat2`.

A simulação é executada até atingir um estado de equilíbrio. Neste trabalho, foi considerado equilíbrio quando a próxima geração calculada é idêntica à geração anterior. Ou seja, o programa para quando:

$$matriz_nova = matriz_anterior$$

Essa comparação é feita pela função `compara_matrizes`, que percorre as 256 posições das duas matrizes e verifica se todos os valores são iguais.

A lógica geral do programa pode ser descrita pelo seguinte pseudo-código:

```
inicio

    desenhar mat1 no Bitmap Display
    esperar 1 segundo

    enquanto verdadeiro:

        calcular proxima geracao:
            origem = mat1
            destino = mat2

        desenhar mat2 no Bitmap Display
        esperar 0,5 segundo

        se mat1 e mat2 forem iguais:
            encerrar simulacao

        calcular proxima geracao:
            origem = mat2
            destino = mat1

        desenhar mat1 no Bitmap Display
        esperar 0,5 segundo

        se mat2 e mat1 forem iguais:
            encerrar simulacao

    esperar 3 segundos
    encerrar programa

fim
```

A chamada de sistema `sleep` foi utilizada com `a7 = 32`, permitindo visualizar cada geração antes da próxima atualização. O programa alterna entre as duas matrizes para evitar a necessidade de copiar manualmente todo o conteúdo de uma matriz para outra a cada geração.

3. Estrutura dos dados

Na seção `.data`, foram declaradas duas matrizes:

```
mat1: .byte ...  
mat2: .byte 0:256
```

A matriz `mat1` contém o estado inicial da simulação. Ela foi preenchida diretamente no código, conforme solicitado no enunciado.

A matriz `mat2` foi inicializada com 256 bytes zerados, pois a matriz possui:

$$16 \times 16 = 256 \text{ células}$$

Como cada célula é armazenada como um byte, cada matriz ocupa 256 bytes de memória.

O cálculo da posição de uma célula `mat[i][j]` é feito por meio da fórmula:

$$posicao = i \times 16 + j$$

Como cada posição da matriz ocupa um byte, o endereço da célula é calculado por:

$$endereco_celula = endereco_base_matriz + posicao$$

No código Assembly, a multiplicação por 16 foi implementada usando deslocamento à esquerda:

```
slli t0, a0, 4
```

Esse comando calcula $i \times 16$, pois deslocar 4 bits à esquerda equivale a multiplicar por 2^4 , ou seja, por 16.

4. Funções implementadas

4.1 readm

A função `readm` possui a seguinte interface:

```
readm(i, j, mat)
```

Parâmetros:

- a0: índice da linha i ;
- a1: índice da coluna j ;
- a2: endereço inicial da matriz.

Retorno:

- a0: valor da célula `mat[i][j]`.

A função `readm` tem como objetivo ler o valor de uma célula da matriz. Antes de acessar a memória, ela verifica se os índices i e j estão dentro dos limites da matriz 16×16 .

As verificações feitas são:

```
i < 0
j < 0
i >= 16
j >= 16
```

Se qualquer uma dessas condições for verdadeira, a função retorna 0. Isso permite tratar automaticamente as bordas da matriz, pois vizinhos fora da matriz são considerados células mortas.

Caso a posição seja válida, a função calcula o deslocamento da célula usando:

$$deslocamento = i \times 16 + j$$

Depois soma esse deslocamento ao endereço inicial da matriz e lê o byte correspondente com a instrução `lb`.

Pseudo-código da função:

```
funcao readm(i, j, mat):

    se i < 0:
        retorna 0

    se j < 0:
        retorna 0

    se i >= 16:
        retorna 0
```

```

se j >= 16:
    retorna 0

deslocamento = i * 16 + j
endereco = mat + deslocamento

retorna memoria[endereco]

```

4.2 write

A função `write` possui a seguinte interface:

```
write(i, j, mat)
```

Parâmetros:

- `a0`: índice da linha `i`;
- `a1`: índice da coluna `j`;
- `a2`: endereço inicial da matriz.

Retorno:

A função não possui retorno.

A função `write` inverte o valor de uma célula da matriz. Caso a célula esteja morta, com valor 0, ela passa a ter valor 1. Caso esteja viva, com valor 1, ela passa a ter valor 0.

Assim como a função `readm`, a função `write` primeiro verifica se os índices `i` e `j` estão dentro dos limites da matriz. Se a posição estiver fora da matriz, a função simplesmente retorna sem alterar nada.

Caso a posição seja válida, a função calcula o endereço da célula, lê seu valor atual, aplica a operação `xori` com o valor 1 e escreve o resultado de volta na memória.

A operação usada foi:

```
xori t1, t1, 1
```

Essa operação inverte o último bit do valor armazenado na célula. Como as células só possuem os valores 0 ou 1, isso transforma:

```

0 em 1
1 em 0

```

Pseudo-código da função:

```
funcao write(i, j, mat):
```

```

se i ou j estiver fora da matriz:
    retorna

deslocamento = i * 16 + j
endereco = mat + deslocamento

valor = memoria[endereco]
valor = valor XOR 1

memoria[endereco] = valor

```

4.3 plotm

A função `plotm` possui a seguinte interface:

```
plotm(mat)
```

Parâmetros:

- `a0`: endereço inicial da matriz.

Retorno:

A função não possui retorno.

A função `plotm` é responsável por desenhar a matriz no Bitmap Display do RARS. Ela percorre todas as posições da matriz 16×16 , verifica se cada célula está viva ou morta, escolhe a cor correspondente e escreve essa cor no endereço correspondente do display.

Foram utilizados dois laços:

- um laço externo para percorrer as linhas `i`;
- um laço interno para percorrer as colunas `j`.

Para cada célula, o programa calcula inicialmente o deslocamento na matriz:

$$deslocamento = i \times 16 + j$$

Como cada posição da matriz ocupa um byte, esse deslocamento é usado diretamente para acessar a célula.

Para o Bitmap Display, cada posição gráfica ocupa uma palavra de 32 bits, ou seja, 4 bytes. Por isso, o deslocamento usado no display é calculado como:

$$deslocamento_display = deslocamento \times 4$$

No Assembly, isso foi feito com:

```
slli t2, t2, 2
```

Depois, o endereço final no display é calculado por:

$$\text{endereco_display} = \text{DISPLAY_BASE} + \text{deslocamento_display}$$

Se a célula lida da matriz for 0, a função escreve a cor `COR_MORTA`. Se for 1, escreve a cor `COR_VIVA`.

Pseudo-código da função:

```
funcao plotm(mat):  
  
    para i de 0 ate 15:  
        para j de 0 ate 15:  
  
            deslocamento = i * 16 + j  
  
            valor = mat[deslocamento]  
  
            deslocamento_display = deslocamento * 4  
            endereco_display = DISPLAY_BASE + deslocamento_display  
  
            se valor == 0:  
                cor = COR_MORTA  
            senao:  
                cor = COR_VIVA  
  
            memoria[endereco_display] = cor
```

4.4 conta_vizinhos

A função `conta_vizinhos` possui a seguinte interface:

```
conta_vizinhos(i, j, mat)
```

Parâmetros:

- `a0`: índice da linha `i`;
- `a1`: índice da coluna `j`;
- `a2`: endereço inicial da matriz.

Retorno:

- **a0**: quantidade de vizinhos vivos da célula `mat[i][j]`.

A função `conta_vizinhos` calcula quantas células vizinhas estão vivas ao redor de uma célula específica.

Cada célula pode ter até oito vizinhos:

<code>[i-1][j-1]</code>	<code>[i-1][j]</code>	<code>[i-1][j+1]</code>
<code>[i][j-1]</code>		<code>[i][j+1]</code>
<code>[i+1][j-1]</code>	<code>[i+1][j]</code>	<code>[i+1][j+1]</code>

Para cada uma dessas posições, a função chama `readm`. Isso simplifica o tratamento das bordas, pois `readm` retorna 0 automaticamente quando os índices estão fora da matriz.

Como essa função chama outra função várias vezes, foi necessário salvar o registrador `ra` na pilha. Além disso, foram usados registradores salvos `s0`, `s1`, `s2` e `s3` para guardar valores que precisavam permanecer válidos durante as chamadas a `readm`.

Os registradores foram usados da seguinte forma:

- `s0`: linha original `i`;
- `s1`: coluna original `j`;
- `s2`: endereço da matriz;
- `s3`: contador de vizinhos vivos.

Antes de retornar, a função restaura todos os registradores salvos.

Pseudo-código da função:

```
funcao conta_vizinhos(i, j, mat):

    contador = 0

    contador += readm(i-1, j-1, mat)
    contador += readm(i-1, j,   mat)
    contador += readm(i-1, j+1, mat)

    contador += readm(i,   j-1, mat)
    contador += readm(i,   j+1, mat)

    contador += readm(i+1, j-1, mat)
    contador += readm(i+1, j,   mat)
    contador += readm(i+1, j+1, mat)

    retorna contador
```

4.5 limpa_matriz

A função `limpa_matriz` possui a seguinte interface:

```
limpa_matriz(mat)
```

Parâmetros:

- `a0`: endereço inicial da matriz.

Retorno:

A função não possui retorno.

A função `limpa_matriz` percorre as 256 posições da matriz e escreve 0 em todas elas.

Essa função é chamada antes de calcular uma nova geração na matriz destino. Isso garante que a matriz destino comece completamente vazia. Assim, durante o cálculo da próxima geração, só é necessário escrever nas posições que devem ficar vivas.

Pseudo-código da função:

```
funcao limpa_matriz(mat):  
  
    para posicao de 0 ate 255:  
        mat[posicao] = 0
```

4.6 proxima_geracao

A função `proxima_geracao` possui a seguinte interface:

```
proxima_geracao(origem, destino)
```

Parâmetros:

- `a0`: endereço da matriz origem;
- `a1`: endereço da matriz destino.

Retorno:

A função não possui retorno.

A função `proxima_geracao` é responsável por aplicar as regras do Jogo da Vida em todas as células da matriz.

Primeiro, ela chama `limpa_matriz` para zerar a matriz destino. Depois percorre todas as posições da matriz origem. Para cada célula, a função:

1. chama `conta_vizinhos` para descobrir quantos vizinhos vivos existem;
2. chama `readm` para saber se a célula atual está viva ou morta;

3. aplica as regras do Jogo da Vida;
4. caso a célula deva estar viva na próxima geração, chama **write** na matriz destino.

Como a matriz destino já está limpa, todas as posições começam com valor 0. Portanto, quando uma célula deve ficar viva, a chamada a **write** inverte o valor de 0 para 1.

As regras aplicadas foram:

```
Se a celula atual esta viva:
    se possui 2 ou 3 vizinhos vivos:
        continua viva
    caso contrario:
        morre

Se a celula atual esta morta:
    se possui exatamente 3 vizinhos vivos:
        nasce
    caso contrario:
        continua morta
```

Pseudo-código da função:

```
funcao proxima_geracao(origem, destino):

    limpa_matriz(destino)

    para i de 0 ate 15:
        para j de 0 ate 15:

            vizinhos = conta_vizinhos(i, j, origem)
            estado_atual = readm(i, j, origem)

            se estado_atual == 1:
                se vizinhos == 2 ou vizinhos == 3:
                    write(i, j, destino)

            senao:
                se vizinhos == 3:
                    write(i, j, destino)
```

4.7 compara_matrizes

A função `compara_matrizes` possui a seguinte interface:

```
compara_matrizes(matA, matB)
```

Parâmetros:

- a0: endereço inicial da primeira matriz;
- a1: endereço inicial da segunda matriz.

Retorno:

- a0 = 1: as matrizes são iguais;
- a0 = 0: as matrizes são diferentes.

A função `compara_matrizes` foi implementada para detectar o equilíbrio da simulação. Ela percorre as 256 posições das duas matrizes e compara os valores armazenados em cada posição.

Se algum par de células correspondente for diferente, a função retorna 0, indicando que a simulação ainda não atingiu equilíbrio. Se todas as posições forem iguais, a função retorna 1, indicando que a próxima geração é igual à geração anterior.

Pseudo-código da função:

```
funcao compara_matrizes(matA, matB):  
  
    para posicao de 0 ate 255:  
  
        valorA = matA[posicao]  
        valorB = matB[posicao]  
  
        se valorA != valorB:  
            retorna 0  
  
    retorna 1
```

Essa função é usada no main logo após o cálculo de cada nova geração.

4.8 main

A função `main` controla a execução geral do programa.

Primeiro, ela desenha a matriz inicial `mat1` no Bitmap Display e aguarda 1 segundo para permitir a visualização do estado inicial.

Em seguida, o programa entra em um laço de execução. A cada iteração, ele calcula uma nova geração e compara a matriz recém-calculada com a matriz anterior. Se as

duas matrizes forem iguais, significa que o sistema atingiu um estado de equilíbrio, e o programa encerra a simulação.

A primeira atualização calcula:

```
mat2 = proxima_geracao de mat1
```

Depois, a matriz `mat2` é desenhada na tela e comparada com `mat1`. Se forem iguais, a simulação termina.

Caso ainda sejam diferentes, a segunda atualização calcula:

```
mat1 = proxima_geracao de mat2
```

Depois, a matriz `mat1` é desenhada na tela e comparada com `mat2`. Se forem iguais, a simulação termina. Caso contrário, o laço continua.

Pseudo-código da função `main`:

```
funcao main:

    plotm(mat1)
    sleep(1000 ms)

    enquanto verdadeiro:

        proxima_geracao(mat1, mat2)
        plotm(mat2)
        sleep(500 ms)

        se compara_matrizes(mat1, mat2) == 1:
            sair do loop

        proxima_geracao(mat2, mat1)
        plotm(mat1)
        sleep(500 ms)

        se compara_matrizes(mat2, mat1) == 1:
            sair do loop

    sleep(3000 ms)
    encerrar programa
```

5. Uso da pilha e convenção de chamadas

Algumas funções do programa chamam outras funções internamente. Por exemplo, `conta_vizinhos` chama `readm` oito vezes, e `proxima_geracao` chama `limpa_matriz`, `conta_vizinhos`, `readm` e `write`.

Sempre que uma função chama outra usando `jal`, o registrador `ra` é sobrescrito. Por isso, funções que chamam outras funções precisam salvar `ra` antes de realizar novas chamadas e restaurá-lo antes de retornar.

Além disso, foram usados registradores salvos `s0`, `s1`, `s2`, etc., para armazenar valores importantes que precisavam permanecer válidos após chamadas de função. Seguindo a convenção de chamadas, esses registradores foram salvos na pilha no início da função e restaurados no final.

Exemplo de salvamento:

```
addi sp, sp, -20
sw ra, 16(sp)
sw s0, 12(sp)
sw s1, 8(sp)
sw s2, 4(sp)
sw s3, 0(sp)
```

Exemplo de restauração:

```
lw s3, 0(sp)
lw s2, 4(sp)
lw s1, 8(sp)
lw s0, 12(sp)
lw ra, 16(sp)
addi sp, sp, 20
ret
```

Esse procedimento garante que as funções possam ser chamadas sem destruir valores importantes de quem as chamou.

6. Funcionamento da exibição gráfica

A exibição gráfica foi realizada por meio de escrita direta na memória mapeada do Bitmap Display.

Cada célula da matriz é convertida para uma posição do display. Como o Bitmap Display armazena cada pixel ou célula gráfica como uma palavra de 32 bits, cada posição gráfica ocupa 4 bytes.

Por isso, para uma célula localizada na posição linear:

$$posicao = i \times 16 + j$$

o deslocamento no Bitmap Display é:

$$deslocamento_display = posicao \times 4$$

O endereço final no display é:

$$endereco_display = DISPLAY_BASE + deslocamento_display$$

Se a célula da matriz contém 0, o programa escreve a cor preta. Se contém 1, escreve a cor amarela.

Dessa forma, a função `plotm` redesenha toda a matriz a cada geração.

7. Critério de parada

O critério de parada utilizado foi a comparação entre a geração atual e a próxima geração calculada. A simulação é encerrada quando não há mais alteração entre duas gerações consecutivas.

Formalmente, considerando uma matriz atual A e uma matriz destino B , após calcular:

$$B = proxima_geracao(A)$$

o programa verifica se:

$$A = B$$

Se essa condição for verdadeira, a simulação atingiu um estado estável, pois a aplicação das regras do Jogo da Vida não altera mais a configuração da matriz.

Como o programa alterna entre `mat1` e `mat2`, essa verificação é feita nos dois sentidos:

- depois de calcular `mat2` a partir de `mat1`, compara-se `mat1` com `mat2`;
- depois de calcular `mat1` a partir de `mat2`, compara-se `mat2` com `mat1`.

Esse critério detecta equilíbrios estáticos, isto é, estados em que a matriz permanece igual de uma geração para a próxima.

8. Conclusão

O programa implementa o Jogo da Vida em Assembly RISC-V no simulador RARS. Foram utilizadas duas matrizes 16×16 , armazenadas como bytes na memória. A matriz inicial é definida diretamente na área `.data`, e a evolução das gerações é calculada alternando entre `mat1` e `mat2`.

As funções obrigatórias `readm`, `write` e `plotm` foram implementadas. Além delas, foram criadas funções auxiliares para contar vizinhos, limpar a matriz destino, calcular a próxima geração e comparar duas matrizes.

A saída gráfica foi realizada por meio do Bitmap Display do RARS, configurado com células de 8×8 pixels e janela de 128×128 pixels. A simulação utiliza chamadas de sistema `sleep` para permitir a visualização da evolução entre as gerações.

Diferentemente de uma execução limitada por uma quantidade fixa de gerações, a versão final executa até atingir um estado de equilíbrio estático, detectado quando duas gerações consecutivas são iguais.

Com isso, o programa atende ao objetivo do trabalho: praticar programação Assembly RISC-V, manipulação de matrizes em memória, uso de funções, convenção de chamadas e saída gráfica mapeada em memória.

9. Arquivos entregues

A entrega será realizada em um arquivo compactado identificado com o número de matrícula do aluno. O arquivo compactado conterá:

- relatório de implementação;
- código Assembly: `jogo_da_vida.asm`.